

APPEAL-DESK

A fair appeals desk for modteams.

Structured intake · audit-trail · one-tap decisions · civil replies.

The problem

Appeals live in modmail. Justice doesn't scale that way.

Today, a banned or removed user has one option: write to modmail and hope.

- Mods get an unstructured wall of text — no link back to the action.
- No history. The same user can re-appeal the same ban 6 times.
- No audit trail. "Why did we uphold this?" is unanswerable a month later.
- No civility scaffold. Replies are drafted from scratch under pressure.

The result: inconsistent decisions, mod burnout, and users who feel unheard even when their appeal is fair.

The solution

A dedicated, fair appeals desk — built into the subreddit.

STRUCTURE

Ban or removal happens → the user sees a structured intake form linked to the exact action. No more freeform modmail. No more lost context.

ASSIST

Mods get a queue with full history, deterministic duplicate detection, and templated civil replies. Three buttons: uphold, overturn, ask for more.

AUDIT

Every decision, reply, and state change is logged. GDPR-style erasure preserves a tombstone. Retention is enforced, not just promised.

*AI is assistive only — a tone-softener and triage hint that the mod can ignore.
The product works end-to-end with AI stripped out. This is enforced by `selectProvider()` and tested.*

The user journey

From mod action to civil reply — in five tracked steps.

1

ACTION

Mod bans or removes content. Appeal-Desk snapshots the original removal reason at the moment of action — user can't edit it later.

2

INVITE

User gets a civil modmail with a link to the appeal form. Form is pre-filled with the action context (read-only) plus a reason field.

3

INTAKE

Submission is validated, sanitised, rate-limited, and deduplicated against the user's prior appeals. AI triage is optional, never blocking.

4

REVIEW

Mods see a dashboard with full history, near-duplicate flags, and three one-tap buttons. Each button opens a reply-confirm form.

5

DECIDE

Mod approves the reply. Decision is recorded **before** the reply is sent — transient send failures never roll back the decision.

Architecture

Platform-free core. Devvit-thin shell.

CORE

TypeScript · zero Devvit imports

store · service · concurrency · validation · dedup · templates · retention

AI

Optional provider behind `selectProvider()`

NoopAiProvider is the default — product is fully functional with AI off

SHELL

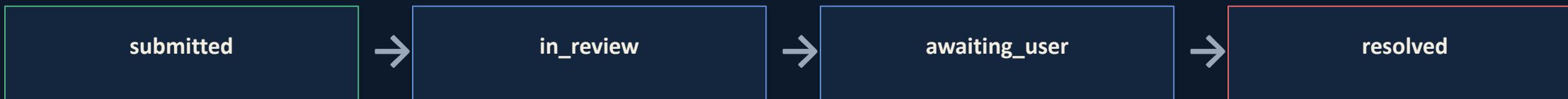
Devvit (.tsx) wiring only

triggers · intake form · dashboard custom post · menu · scheduler

*The platform-free core has 100% test coverage *without* the Devvit runtime. That's the headline architectural call.*

Concurrency · The hard part, done

One ban. One appeal. No matter how many tabs are open.



✓ Atomic action-lock

WATCH/MULTI/EXEC CAS. Two users opening the same appeal in the same millisecond — only one wins. Proven by parallel-execution tests.

✓ Record-then-send

Decision is persisted **before** the reply is dispatched. A modmail send failure throws `REPLY_DELIVERY_FAILED` but the decision stands.

✓ Terminal-state guard

A ``resolved`` appeal rejects further decisions with `INVALID_STATE_TRANSITION`. No accidental over-write of a final verdict.

✓ Tie-safe pagination

Cursor is ``{score, id}``, not a bare score. Same-millisecond appeals no longer skip pages — fix landed in BUILD 003.

The AI stance

AI drafts. Humans decide. The product survives AI removal.

Three rules, enforced structurally:

1 · AI never blocks intake

`submitAppeal()` calls the triage provider best-effort. Failure returns null. The appeal is created either way. Tested with a failing provider.

2 · The mod always confirms

Every one-tap decision opens a reply-confirm form. The drafted reply is editable. AI output is clamped (empty / >4× length → fallback to template).

3 · NoopAiProvider is the default

`selectProvider(aiEnabled, backend)` returns a no-op when AI is off or no backend is wired. The full happy path works without an API key.

*Why this matters: a tool that **needs** AI fails closed if the model has a bad day. Appeal-Desk fails open — a slightly less polished reply, but the workflow still works.*

Proof, not promises

Numbers from the actual repo. Reproducible on your machine.

288 / 288

tests pass

99.97 %

line coverage

100 %

function coverage

99.06 %

branch coverage

18

test files

0

TypeScript errors

0

ESLint issues

4

build commits on main

```
`npm ci && npm run lint && npx tsc --noEmit && npx vitest run --coverage`
```

Honest defect log

Reviewed end-to-end. Each defect found — and fixed.

F1 · HIGH	<code>`npm run lint` broken</code>	<i>No ESLint config existed.</i>	Added <code>`.eslintrc.cjs`</code> + deps. Clean.
F2 · MED	Same-ms cursor tie-skip	<i><code>`cursor - 1`</code> dropped co-scored entries.</i>	Tuple cursor <code>{score, id}</code> . New test.
F3 · MED	Unbounded queue read	<i><code>`openQueuePage`</code> hydrated whole index.</i>	<code>`limit: {offset, count}`</code> everywhere.
F4 · LOW	Seed key smuggled	<i><code>`actionLock`</code> builder reused.</i>	First-class <code>keys.actionSeed()</code> .
F5 · LOW	<code>purgeExpired</code> over-fetch	<i>Symmetric with F3.</i>	Same bounded-read treatment.
F6 · NIT	Stray <code>`ms`</code> field	<i>Type cast hid a tiny lie.</i>	Dropped the field and the cast.
Wiring	<code>syncConfig</code> only on install	<i>Settings edits ignored till reinstall.</i>	Now runs on submission + AppUpgrade.
Wiring	Retention was dead code	<i><code>purgeExpired/redactAppeal</code> never called.</i>	Daily scheduler job + service entry points.

Beyond the MVP

What ships next — width first, then depth.

Width (high-leverage modules)

- core/analytics — overturn rate, time-to-decision
- core/policy — eligibility + cooldowns per sub
- core/notifications — user-side status updates
- core/escalation — multi-mod sign-off
- core/export — CSV/JSON portability
- core/i18n — message catalogs per sub language

Depth (richer existing modules)

- Bulk decisions on near-duplicates
- Incremental dedup (MinHash, no full re-hydration)
- Conditional templates + token linter
- PII pre-flags + Unicode normalisation in dedup
- Correlation IDs in observability
- AI eval harness with golden fixtures

Install · Try it

Two commands. Live on your test sub in 90 seconds.

From source:

```
$ git clone https://github.com/vsenthil7/appeal-desk && cd appeal-desk  
$ npm ci && devvit upload && devvit install r/your_test_sub
```

From the App Directory:

developers.reddit.com/apps/appeal-desk → Install on subreddit

Permissions requested:

- reddit — read mod actions, send modmail (mod scope)
- redis — store appeals, history, locks, indexes
- scheduler — SLA nudges + daily retention purge
- http: false — fully on-platform, no external services

Appeal-Desk

A fair appeals desk for modteams.

AI assists. Humans decide. The audit trail proves it.

github.com/vsenthil7/appeal-desk

MIT · 288/288 tests · 99.97 % coverage · 4 BUILD commits · Devvit App Directory

Thank you.